

LangChain for Beginners

A simple, no-overengineering study guide: learn the basics, build one small project, then move to RAG and agents.

1. What is LangChain?

Simple idea: LangChain is a Python framework that helps you connect an AI model to prompts, tools, documents, and application logic. It gives you common interfaces so you can build AI applications without writing every integration yourself.

Think of it like this: **Your Python app** → **LangChain** → **AI model**. Later you can add tools or your own documents.

2. The 6 things to learn first

- **Model** — the AI that generates the answer.
- **Prompt** — instructions and user input sent to the model.
- **Messages** — a structured way to send system/user/assistant conversation.
- **Tool** — a Python function the model can ask your application to run.
- **Document** — text plus metadata, useful when working with files.
- **Agent** — a model that can decide when to use available tools.

3. Do NOT learn everything at once

For your first week, ignore LangGraph, middleware, deployment, complex memory systems, vector databases, and advanced agent architectures. First understand model → prompt → response, then one tool, then documents/RAG.

4. Your first project: Simple AI Assistant

Goal: make a Python program that sends a question to an AI model and prints the answer. After that, add a tiny calculator tool.

Setup

Use Python 3.10+. Create a folder, create a virtual environment, activate it, and install LangChain plus the provider package. The official docs currently recommend installing LangChain with pip and using separate provider integration packages.

```
# Windows
mkdir langchain-beginner
cd langchain-beginner
python -m venv .venv
.venv\Scripts\activate

# macOS / Linux
mkdir langchain-beginner
cd langchain-beginner
python3 -m venv .venv
source .venv/bin/activate

# Install
pip install -U langchain langchain-openai
```

Set your API key

```
# Windows PowerShell
$env:OPENAI_API_KEY="your_api_key_here"

# macOS / Linux
export OPENAI_API_KEY="your_api_key_here"
```

Create app.py

```
from langchain.chat_models import init_chat_model
```

```

model = init_chat_model("openai:gpt-5.4")
question = input("Ask me something: ")
response = model.invoke(question)
print("\nAI:", response.content)

```

What happens? `input()` gets your question → `model.invoke()` sends it to the model → `response.content` contains the answer.

5. Add your first tool

A tool is simply a function that the agent can call. Start with something easy, such as a calculator.

```

from langchain.agents import create_agent

def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[add],
    system_prompt="You are a helpful assistant. Use the add tool when useful."
)

result = agent.invoke({
    "messages": [
        {"role": "user", "content": "What is 25 + 17?"}
    ]
})

print(result["messages"][-1].content)

```

The important idea is not memorizing the code. Understand that the model can decide to use your Python function as a tool and then use its result.

6. Learn Documents and RAG next

RAG means Retrieval-Augmented Generation. In simple words: instead of asking the AI to rely only on its built-in knowledge, your application finds relevant pieces of your own documents and gives those pieces to the model before it answers.

Basic RAG picture:

```

PDF / notes
↓
Load text
↓
Split into smaller chunks
↓
Create embeddings
↓
Store/search chunks
↓
Find relevant chunks for the question
↓
Send question + relevant chunks to the model
↓
Answer

```

Do this only after you are comfortable with the basic model call and tools. The official LangChain semantic-search tutorial uses documents, text splitters, embeddings, vector stores, and retrievers.

7. A simple 14-day learning plan

- Days 1–2: Python basics — functions, lists, dictionaries, imports, virtual environments.
- Day 3: Make the first LangChain model call.
- Day 4: Learn prompts and messages.
- Day 5: Build a small command-line chatbot.

- Day 6: Learn what tools are.
- Day 7: Build the calculator-tool agent.
- Days 8–9: Learn Document, document loaders, and text splitting.
- Days 10–11: Learn embeddings and vector stores.
- Days 12–13: Build a PDF question-answering RAG app.
- Day 14: Review and clean your project. Do not add advanced features yet.

8. Beginner vocabulary

LLM / Chat model — The AI model that produces language responses.

Prompt — Instructions or input given to the model.

Tool — A function the model can request your program to execute.

Agent — A model-driven loop that can choose tools and perform steps.

Document — Text plus metadata used by LangChain document workflows.

Embedding — A numeric representation of text used for similarity search.

Vector store — A system that stores embeddings and helps find similar text.

Retriever — The component that finds relevant documents/chunks.

RAG — Retrieve useful information first, then ask the model to answer using it.

9. Official LangChain resources

Overview

<https://docs.langchain.com/oss/python/langchain/overview>

Install

<https://docs.langchain.com/oss/python/langchain/install>

Quickstart

<https://docs.langchain.com/oss/python/langchain/quickstart>

Learn / Tutorials

<https://docs.langchain.com/oss/python/learn>

Semantic Search / RAG with a PDF

<https://docs.langchain.com/oss/python/langchain/knowledge-base>

Python API Reference

<https://docs.langchain.com/oss/python/reference/overview>

10. The learning rule

Read a little → code a little → break it → fix it → explain it in your own words.

Do not try to memorize LangChain. Learn the purpose of each component and build small projects. When you can explain model, prompt, tool, document, embedding, retriever, RAG, and agent in simple words, you have a strong beginner foundation.

Note: LangChain changes quickly. Use this PDF for concepts and the linked official documentation for current package names, model names, and APIs.